

Reinforcement Learning

- Markov Decision Process (MDP)

$(S, A, \{P_{sa}\}, \gamma, R)$

S : Set of states

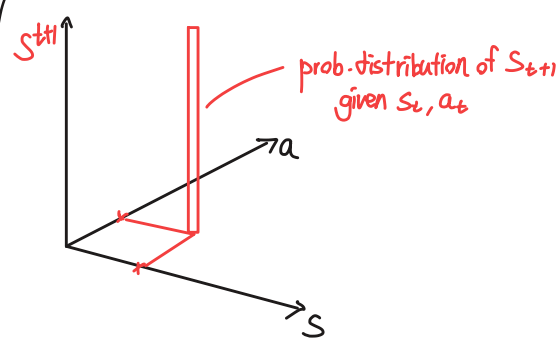
A : Set of actions

P_{sa} : State transition probabilities

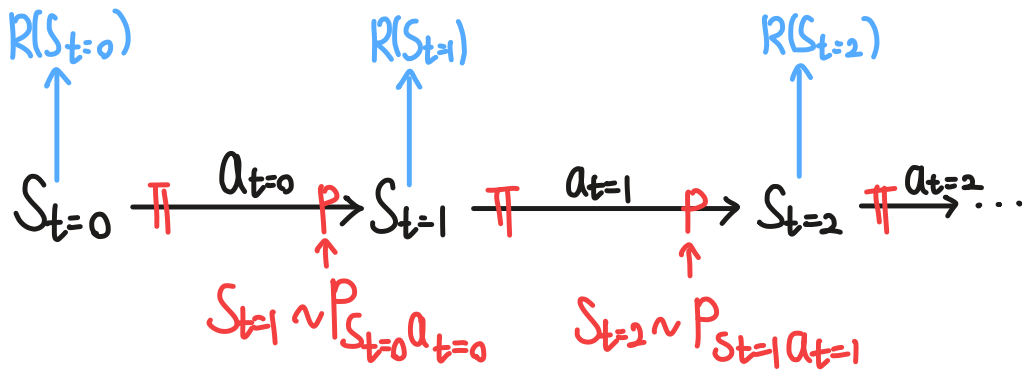
γ : discount factor $\in [0, 1)$

R : Reward function $S \times A \mapsto \mathbb{R}$
or $S \mapsto \mathbb{R}$

$\{P_{sa}\} \approx$ Laws of nature



- Policy $\pi: S \rightarrow A$ ($\pi(s_i) = a_i$)



1. Ideal Setting: S, A are finite

Given π and start state s_i , we can construct $V^\pi(s_i)$. (Value function)

$V^\pi(s_i)$ = Expected value of: sum of all future discounted rewards

$$= E_{\gamma \sim p(\cdot | s_{t=0} = s_i; \pi)} \left[\sum_{j=0}^{\infty} \gamma^j R(s_{t=j}) \right]$$

// Bellman Equation

$$= R(s_i) + \gamma \sum_{s' \in S} P_{s_i \pi(s_i)}(s') V^\pi(s')$$

$$= R(s_i) + \gamma E_{s' \sim P_{s_i \pi(s_i)}} [V^\pi(s')]$$

$$\sim Q^\pi(s_i, a_i) = R(s_i) + \gamma \sum_{s' \in S} P_{s_i a_i}(s') V^\pi(s')$$

⇒ Matrix Notation

$$P_{s_i \pi(s_i)} \in [0, 1]^{|S|} = (0.00x, 0.00x, \dots)^T$$

s.t. $\sum_{s'} P_{s_i \pi(s_i)}(s') = 1$

$$P^\pi \xrightarrow{\text{stack}} \in [0, 1]^{|S| \times |S|} = \begin{bmatrix} \text{---} P_{s_1 \pi(s_1)} \text{---} \\ \vdots \\ \text{---} P_{s_k \pi(s_k)} \text{---} \\ \vdots \end{bmatrix}$$



$$\vec{V} = \vec{R} + \gamma P^\pi \vec{V}$$

$$(I - \gamma P^\pi) \vec{V} = \vec{R}$$

$$\vec{V} = (I - \gamma P^\pi)^{-1} \vec{R} \quad // \text{Going from } \pi \text{ to } \vec{V}$$

$$R \in \mathbb{R}^{|S|} = (R(s_1), R(s_2), \dots)^T$$

$$V^\pi \in \mathbb{R}^{|S|} = (V^\pi(s_1), V^\pi(s_2), \dots)^T$$

⊕ Estimating P_{sa} when not given

P_{sa} is not given ⇒ Estimate State Transition Probabilities

$$S_{t=0} \xrightarrow{a_{t=0}} S_{t=1} \xrightarrow{a_{t=1}} S_{t=2} \longrightarrow \dots$$

run many times, extract triplets $(s_{\text{prev}}, a_{\text{prev}}, s_{\text{next}})$

$$- P_{sa}(s') = \frac{\# \text{ times we took action } a \text{ in state } s \text{ and got to state } s'}{\# \text{ times we took action } a \text{ in state } s}$$

(This requires exhaustive search, so we instead initialize probabilities uniformly and increment the counter whenever we encounter them)

* Proof of the Bellman equation:

Given $(S = \{s_1, s_2, s_3\}, A, \{P_{sa}\}, r, R)$ with π fixed:

$$V^\pi(s_1) = E_{\gamma \sim p(\cdot | s_{t=0} = s_1, \pi)} [R(s_{t=0}) + \gamma R(s_{t=1}) + \gamma^2 R(s_{t=2}) + \dots]$$

$$= R(s_1) + E_{\gamma \sim p(\cdot | s_{t=0} = s_1, \pi)} [\gamma R(s_{t=1}) + \gamma^2 R(s_{t=2}) + \dots]$$

$$= R(s_1) + E_{\gamma \sim p(\cdot | s_{t=0} = s_1, \pi)} \left[\sum_{k=1}^{\infty} \gamma^k R(s_{t=k}) \right]$$

$$= R(s_1) + E_{\gamma \sim p(\cdot | s_{t=0} = s_1, \pi)} \left[\gamma \sum_{k=1}^{\infty} \gamma^{k-1} R(s_{t=k}) \right]$$

$$= R(s_1) + E_{\gamma \sim p(\cdot | s_{t=0} = s_1, \pi)} \left[\gamma \sum_{u=0}^{\infty} \gamma^u R(s_{t=u+1}) \right]$$

$$= R(s_1) + \gamma E_{\gamma \sim p(\cdot | s_{t=0} = s_1, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u+1}) \right]$$

↓ ①

$$= R(s_1) + \gamma \left(P_{s_1 \pi(s_1)}(s_1) \cdot E_{\gamma \sim p(\cdot | s_{t=0} = s_1, s_{t=1} = s_1, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u+1}) \right] \right. \\ \left. + P_{s_1 \pi(s_1)}(s_2) \cdot E_{\gamma \sim p(\cdot | s_{t=0} = s_1, s_{t=1} = s_2, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u+1}) \right] \right. \\ \left. + P_{s_1 \pi(s_1)}(s_3) \cdot E_{\gamma \sim p(\cdot | s_{t=0} = s_1, s_{t=1} = s_3, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u+1}) \right] \right)$$

↓ ②

$$= R(s_1) + \gamma \left(P_{s_1 \pi(s_1)}(s_1) \cdot E_{\gamma \sim p(\cdot | \cancel{s_{t=0} = s_1}, s_{t=1} = s_1, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u+1}) \right] \right. \\ \left. + P_{s_1 \pi(s_1)}(s_2) \cdot E_{\gamma \sim p(\cdot | \cancel{s_{t=0} = s_1}, s_{t=1} = s_2, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u+1}) \right] \right. \\ \left. + P_{s_1 \pi(s_1)}(s_3) \cdot E_{\gamma \sim p(\cdot | \cancel{s_{t=0} = s_1}, s_{t=1} = s_3, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u+1}) \right] \right)$$

↓ ③

$$= R(s_1) + \gamma \left(P_{s_1 \pi(s_1)}(s_1) \cdot E_{\gamma \sim p(\cdot | s_{t=0} = s_1, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u}) \right] \right. \\ \left. + P_{s_1 \pi(s_1)}(s_2) \cdot E_{\gamma \sim p(\cdot | s_{t=0} = s_2, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u}) \right] \right. \\ \left. + P_{s_1 \pi(s_1)}(s_3) \cdot E_{\gamma \sim p(\cdot | s_{t=0} = s_3, \pi)} \left[\sum_{u=0}^{\infty} \gamma^u R(s_{t=u}) \right] \right)$$

$$= R(s_1) + \gamma \left(P_{s_1 \pi(s_1)}(s_1) \cdot V^\pi(s_1) + P_{s_1 \pi(s_1)}(s_2) \cdot V^\pi(s_2) + P_{s_1 \pi(s_1)}(s_3) \cdot V^\pi(s_3) \right)$$

① Law of total expectation

② Markov Property #memoryless

(In a prob. sequence, the next state only depends on the current state & actions)

③ Stationarity assumption

(P_{sa} doesn't depend on t)

So, given $(S, A, \{P_{sa}\}, \gamma, R)$ and a fixed policy π ,
we know all the value function outputs of all states.

Now, our main question is: How do we find the best π ?

We define the best π as π that $V^\pi(s_i)$ is the maximum across all $s_i \in S$. (Yes, this is possible)

and call it the optimal policy: $\pi^* = \underset{\pi}{\operatorname{argmax}} V^\pi(s_i) \quad \forall s_i \in S$

And we call the value function at π^* the optimal value function.

$$V^{\pi^*}(s_i) = \max_{\pi} V^\pi(s_i) \quad \forall s_i \in S$$

($= V^*(s_i)$ # universal)

However the optimal policy above should be obtained by exhaustive search (by its definition)

So we developed a bypass (value iteration, policy iteration)

Value iteration (we don't touch policies during iteration)

for all $s_i \in S$, $V(s_i) := 0$

Repeat until convergence ϵ

change of $V < \epsilon$

for all $s_i \in S$:

$$V(s_i) := R(s_i) + \max_{a_j \in A} \sum_{s' \in S} P_{s_i a_j}(s') V(s')$$

// Bellman backup operator, Guaranteed to converge

}

// Result is V^*

for all $s_i \in S$:

$$\pi^*(s_i) = \underset{a_j \in A}{\operatorname{argmax}} \sum_{s' \in S} P_{s_i a_j}(s') V^*(s')$$

Policy Iteration

Randomly initialize π

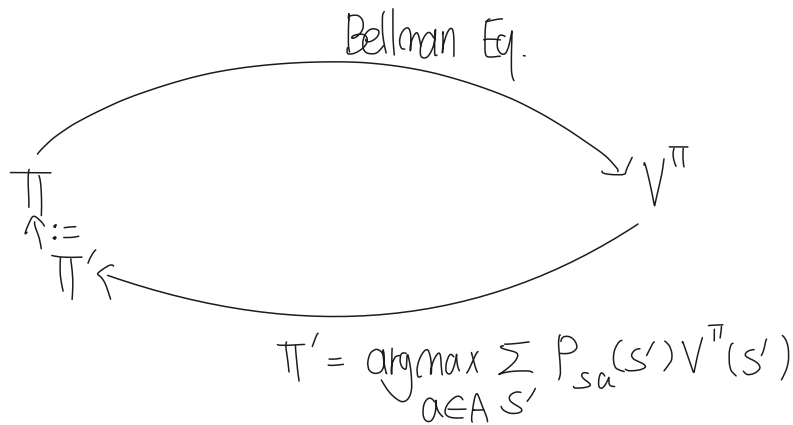
Repeat until convergence Σ

$$\vec{V} = (I - \gamma P^\pi)^{-1} \vec{R} \quad // \text{ policy evaluation}$$

for all $s_i \in S$:

$$\pi(s_i) := \underset{a \in A}{\operatorname{argmax}} \sum_{s' \in S} P_{s_i a}(s') V^\pi(s') \quad // \text{ policy improvement}$$

}



Convergence: when $\pi = \pi'$ (strict)

So we get π^* , V^* exactly

⊕ Pro: gets exactly π^* , V^* (V.I only gets π^* exactly right)

Con: has to calculate the inverse

↪ We can use value iter. to get close and policy iter. to snap into the exact solution

② S, A are not finite (continuous) // Real World Setting

→ We can try to solve it with discretization, but we would lose generalization

→ Also if we were to discretize high dimension space, we get hit w/ curse of dimensionality

∴ Discretization doesn't work in practice $\Rightarrow$ Approximation

We approximate $V(s)$ with $\Theta^T \phi(s)$, where Θ is a learnable parameter and ϕ is the feature mapping

⊕ Estimating P_{sa} when not given

n examples (trials), each example can take up to T steps

$$\begin{pmatrix} S_0^{(1)} \longrightarrow S_1^{(1)} \longrightarrow S_2^{(1)} \longrightarrow \dots \\ S_0^{(2)} \longrightarrow S_1^{(2)} \longrightarrow S_2^{(2)} \longrightarrow \dots \\ \vdots \end{pmatrix}$$

$$\hat{A}, \hat{B} = \underset{A, B}{\operatorname{argmin}} \sum_{i=1}^n \sum_{k=0}^{T-1} \|S_{t=k+1}^{(i)} - (AS_{t=k}^{(i)} + Ba_{t=k}^{(i)})\|_2^2$$

$$S_{t=k+1}^{(i)} - (\hat{A} S_{t=k}^{(i)} + \hat{B} a_{t=k}^{(i)}) = \epsilon_{t=k}^{(i)}, \quad \hat{\Sigma} = \frac{1}{nT} \sum_{i=1}^n \sum_{j=0}^{T-1} \epsilon_{t=j}^{(i)} \epsilon_{t=j}^{(i)T}$$

$$P_{sa} = N(\hat{A}s + \hat{B}a, \hat{\Sigma}), s' \sim P_{sa}$$

(We're assuming the framework of:

$$S_{t=k+1} = AS_{t=k} + B \cdot a_{t=k} + \epsilon_t, \quad \epsilon_{t=k} \sim N(0, \Sigma)$$

$$\text{So, } S_{t=k+1} \sim N(AS_{t=k} + Ba_{t=k}, \Sigma)$$

In finite settings + known (or estimable) P_{sa} and $r < 1$, $v \cdot i$ and $p \cdot i$ give us π^* exactly (and V^* exactly for $p \cdot i$, approx. for $v \cdot i$). In continuous settings the tabular machinery breaks: we can't store V as a vector, can't enumerate states, can't sum over s' . Special structure can still be solved exactly, but in general we approximate those three (function approx., sampling states, sampling next states) using various algorithms.

Fitted Value Iteration

(Assumption: A is discrete, S is continuous and each $s_i \in \mathbb{R}^d$, can sample from $P_{s,a}$)

1) Randomly Sample n states $s_1, s_2, \dots, s_n \in S$ // sampling states

2) $\theta := 0$

3) Repeat: ϵ

for all $s_i \in S_n$

for all $a \in A$:

sample from $P_{s_i,a}$: $s_{i1}, s_{i2}, \dots, s_{ik} \sim P_{s_i,a}$

$$q(a) := \frac{1}{k} \sum_{j=1}^k (R(s_i) + \gamma V(s'_{ij}))$$

// sampling next states

$$y_i := \max_a q(a) \quad // \approx R(s_i) + \max_{a \in A} \gamma \sum_{s' \in S} P_{s_i,a}(s') V(s')$$

$$\theta := \underset{\theta}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^n (\theta^T \phi(s_i) - y_i)^2 \quad // \text{function approximation}$$

* Unlike V.I where you do $\pi^*(s_i) = \underset{a_j \in A}{\operatorname{argmax}} \sum_{s' \in S} P_{s_i,a_j}(s') V^*(s')$
for every $s_i \in S$, you don't calculate π^* upfront here.

Instead when actually running, every step (state) we sample k s' s

$$\text{and calculate } \pi(s) = \underset{a}{\operatorname{argmax}} \frac{1}{k} \sum_{j=1}^k \theta^T \phi(s'_j) \quad (s'_j \sim P_{s,a})$$

(yes, we search over every a every step.)

* Not guaranteed to converge

Policy Gradient (Given now the policy is also random, what's the best policy?) // REINFORCE

change 1: $\pi: S \rightarrow A \rightarrow \pi_\theta(a|s)$

change 2: Finite time horizon T (how many steps)

change 3: Don't need to know P_{sa} , just need to be able to sample from it

Let $\tau = (s_0, a_0, \dots, s_{T-1}, a_{T-1}, s_T)$ // full trajectory

$s_0 \sim \mu$ // s_0 is sampled by some distribution

$\pi_\theta(a|s)$ // prob. distribution of a given s (usually a neural network)

$P_\theta(\tau)$ // prob. distribution of the resulting total trajectory using $\pi_\theta(a|s)$

$s_t \sim P_{s_{t-1}, a_{t-1}}$ // sampling s_t from an unknown distribution $P_{s_{t-1}, a_{t-1}}$

$\eta(\theta) = E_{\tau \sim P_\theta} \left[\sum_{t=0}^{T-1} \gamma^t R(s_t) \right] := E[f(\tau)]$ // maximize this with gradient ascent

$\hat{\theta} = \underset{\theta}{\operatorname{argmax}} \eta(\theta) = \underset{\theta}{\operatorname{argmax}} E_{\tau \sim P_\theta} [f(\tau)]$

$$\nabla_\theta E_{\tau \sim P_\theta} [f(\tau)] = \nabla_\theta \int_{\tau} P_\theta(\tau) f(\tau) d\tau = \int_{\tau} \nabla_\theta [P_\theta(\tau) f(\tau)] d\tau = \int_{\tau} \nabla_\theta [P_\theta(\tau)] \cdot f(\tau) d\tau$$

$$\left(\text{Since } \nabla_\theta \log P_\theta(\tau) = \frac{\nabla_\theta P_\theta(\tau)}{P_\theta(\tau)}, \quad \nabla_\theta P_\theta(\tau) = \nabla_\theta [\log P_\theta(\tau)] \cdot P_\theta(\tau) \right)$$

$$= \int_{\tau} P_\theta(\tau) \cdot \nabla_\theta \log P_\theta(\tau) \cdot f(\tau) d\tau = E_{\tau \sim P_\theta} [\nabla_\theta \log P_\theta(\tau) \cdot f(\tau)]$$

$$\left(\begin{array}{l} P_\theta(\tau) = \mu(s_0) \cdot \pi_\theta(a_0|s_0) \cdot P_{s_0 a_0}(s_1) \cdot \pi_\theta(a_1|s_1) \cdot P_{s_1 a_1}(s_2) \cdots \\ \log P_\theta(\tau) = \log \mu(s_0) + \log \pi_\theta(a_0|s_0) + \log P_{s_0 a_0}(s_1) + \log \pi_\theta(a_1|s_1) + \log P_{s_1 a_1}(s_2) \cdots \\ \nabla_\theta \log P_\theta(\tau) = \nabla_\theta (\log \pi_\theta(a_0|s_0) + \log \pi_\theta(a_1|s_1) + \dots) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \end{array} \right)$$

$$= E_{\tau \sim P_\theta} \left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right) \cdot f(\tau) \right] = E_{\tau \sim P_\theta} \left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right) \cdot \left(\sum_{t=0}^{T-1} \gamma^t R(s_t) \right) \right]$$

* In real life we approximate by sampling $\tau^{(1)}, \dots, \tau^{(n)}$

$$\left(E_{\tau \sim P_\theta} [\nabla_\theta \log P_\theta(\tau) \cdot f(\tau)] \approx \frac{1}{n} \sum_{i=1}^n \nabla_\theta \log P_\theta(\tau^{(i)}) \cdot f(\tau^{(i)}) \right)$$

// Some Extensions

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}} \left[\left(\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \cdot \left(\sum_{t=0}^{T-1} \gamma^t R(s_t) \right) \right]$$

// Problem: multiplying by the whole $f(\tau)$ drags in rewards before t .

those are already fixed regardless of a_t , so zero info for the gradient, just pure added noise $\Rightarrow$ high variance

change: causality (later action can't affect earlier reward)

$$E[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot R(s_j)] = 0 \text{ for } j < t,$$

so distribute the product over t and drop those terms (keep $\sum_{j \geq t}$)

$\Rightarrow$ variance down, no bias since we dropped those expectation 0

$$= \sum_t E_{\tau \sim p_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \gamma^t \sum_{j \geq t} \gamma^{j-t} R(s_j) \right]$$

// problem: if all remaining returns are positive, every action's probability gets pushed up and learning comes only from relative magnitudes, so variance is still high

// change: subtract a baseline $B(s_t)$ (valid since it doesn't depend on a_t)

$$E_{a_t \sim \pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] B(s_t)$$

$$= B(s_t) \nabla_{\theta} \sum_a \pi_{\theta}(a | s_t) = B(s_t) \nabla_{\theta} 1 = 0$$

so expectation is unchanged, now we learn from

"was this better than average"

$$= \sum_{t=0}^{T-1} E_{\tau \sim p_{\theta}} \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot \gamma^t \left(\sum_{j \geq t} \gamma^{j-t} R(s_j) - B(s_t) \right) \right]$$

// Problem: $B(s_t)$ is still unspecified, any choice is unbiased but how much variance it removes depends on the choice

// Change: $B(s_t) := V^{\pi_\theta}(s_t)$ (the expected return from that state)

$$= \sum_{t=0}^{T-1} \mathbb{E}_{\tau \sim p_\theta} \left[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot r^t \left(\underbrace{\sum_{j=t}^{T-1} \gamma^{j-t} R(S_j)}_{\text{"Advantage"}} - V^{\pi_\theta}(s_t) \right) \right]$$

how much better than average the action was

$$\cdot \sum_{j=t}^{T-1} \gamma^{j-t} R(S_j) \approx Q^{\pi_\theta}(s_t, a_t)$$

• Learning V alongside $\rightarrow$ actor-critic

RL for LLMs

Agentic AI: LLM + Tools (can act) for LLMs to do things (1. needed to be trained to act, 2. need tools)

RL HF / RL AIF / RL VR * our focus train separate reward model

MDP for LM:

• $S_t : (\underbrace{x, a_0, \dots, a_{t-1}}_{\substack{\text{initial external} \\ \text{input tokens}}})$ tokens the model generated so far

• A : set of vocabulary V

• $\{P_{sa}\}$: deterministic: $S_{t+1} = S_t \parallel a_t = (x, a_0, a_1, \dots, a_{t-1}, a_t)$
(concatenate)

so when $S' = S \parallel a$,

$$P_{sa}(S') = 1$$

$$P_{sa}(S) = 0 \quad S \neq S'$$

• Reward: $\text{Verifier}(x, y) \in \mathbb{R}$

$$\text{ex) } \text{Ver}(x, y) = \begin{cases} 1 & \text{if response passes verifier} \\ 0 & \text{otherwise} \end{cases}$$

$$R(S_t) = 0 \quad t < T$$

$$R(S_T) = \text{Ver}(x, y) \quad \text{once you completed all generated}$$

• $\gamma = 1$ (realistic settings - finite T)

Random Policy $\pi_\theta(a_t | S_t)$

$$y = (a_0, \dots, a_{T-1}), \quad \pi_\theta(y | x) = \prod_{t=0}^{T-1} \pi_\theta(a_t | S_t) \Rightarrow \log \pi_\theta(y | x) = \sum_{t=0}^{T-1} \log \pi_\theta(a_t | S_t)$$

* special stopping tokens - end_turn

Agentic Setting

special stopping tokens — end_turn

tool_call

$$S_t : (x, z_0, z_1, \dots, z_{t-1})$$

z_t : next token

(= a in non-agentic settings)

z / typical vocabulary (a) tokens

tool call tokens

$(c) : (x, \langle fn-name \rangle get_weather, lat, lon \langle /fn-name \rangle Tool_call$

$P_{S_t z_t}$: append z_t to S_t

$R = Ver$

$\gamma = 1$

$$\pi_{\theta}(z_t | S_t) = LLM$$

$$\text{Objective: } \max_{\theta} \eta(\theta) = \max_{\theta} E_{\gamma \sim P_{\theta}} [R(\gamma)]$$

toolkit

```
get_weather(lat, lon) {  
  .....  
  .....  
}
```

...

Final Trajectory $\gamma = x, a, a, a, \dots a, c_1, o_1, a, \dots a, c_J, o_J, a, \dots a$

LLM first call output — done by harness

if (LLM output.endsWith(tool_call)) {

$O =$ execute the tool according to c

(could be deterministic ex. calculator
or stochastic: $O \sim p_{tool}(\cdot | S, c)$
ex. weather.com)

$S' = S || O$

(append output)

LLM.call with s'

} else {

end generation and present to user

}

(Policy Gradient Process)

$M(\gamma) = C_s$ and $a_s (= z_s)$ // not o_s

$$P_{\theta}(\gamma|q) = \prod_{t \in M(\gamma)} \pi_{\theta}(z_t | s_t) \times \prod_{j \in \text{output gen}} P_{\text{tool}}(o_j | s_j, c_j)$$

$$\text{Then } \nabla_{\theta} \log P_{\theta}(\gamma | x) = \sum_{t \in M(\gamma)} \nabla_{\theta} \log \pi_{\theta}(z_t | s_t).$$

$$\nabla_{\theta} J(\theta) = E \left[\sum_{t \in M(\gamma)} \nabla_{\theta} \log \pi_{\theta}(z_t | s_t) (G_t - B(s_t)) \right]$$

ex) $x \rightarrow a_1 \rightarrow a_2 \rightarrow \text{tool-call} \rightarrow o_1 \rightarrow a_3 \rightarrow \text{tool-call} \rightarrow o_2 \rightarrow a_3 \rightarrow \text{end-turn}$

$\Rightarrow$ only the checkmarks contribute to loss.

Axis-1 Discrete vs Continuous
α) P.I, V.I α) F.V.I

Axis-2 Focus on Value function vs Focus on Policy
ex) V.I, F.V.I, Q.L ex) P.I, P.G, PPO, GRPO

Axis-3 Model based vs Model Free
(Are we explicitly modeling P_{sa}) // F.V.I can be both
ex) P.I, V.I ex) P.G, Q.L

Axis-4 On-policy vs Off-policy
(Learning by doing it yourself, (Learning by looking at other agents or one's past)
following your own policy) behavior policy $\neq$ target policy
P.G